VisionInspect AI

AI-Powered Manufacturing Defect Detection & Quality Inspection System

Database Design & Implementation Documentation

PostgreSQL · SQLAlchemy 2.0 ORM · FastAPI Backend

Prepared for Final Year Project Documentation, Internship Submission,
Project Review & Mentor Presentation

Akshaya Reddy

July 2026

Table of Contents

1. Database Overview

VisionInspect AI persists all operational data — operator accounts, uploaded product images, AI inspection results, and generated compliance reports — in a PostgreSQL relational database, accessed through the SQLAlchemy 2.0 ORM inside the FastAPI backend. This section explains the reasoning behind every core technology decision.

1.1 Why PostgreSQL

- **Relational-first data:** Strong relational integrity — the project's data is inherently relational (a user uploads many images, each image undergoes an inspection, each inspection generates a report), so foreign keys, cascades, and joins are first-class requirements rather than an afterthought.
- **ACID compliance:** PostgreSQL enforces ACID transactions natively, which matters for an inspection pipeline where an image row, an inspection row, and a report row must stay consistent even under concurrent uploads.
- **Rich data types:** Native support for ENUM types, JSONB, full-text search, and rich indexing (B-Tree, GIN) gives room to grow — e.g. storing raw model output as JSONB later — without switching engines.
- **Scalability & cost:** Open-source, horizontally and vertically scalable, and battle-tested for production workloads at manufacturing scale.
- **Ecosystem fit:** First-class support in SQLAlchemy 2.0, psycopg driver maturity, and predictable behavior under FastAPI's async/sync session patterns.

1.2 Why SQLAlchemy ORM

- **Type-safe schema:** Database tables are defined as typed Python classes (Mapped[int], Mapped[str]), so schema and application code cannot silently drift apart.
- **Object-relational mapping:** relationship() and back_populates let Python code navigate user.uploaded_images or image.inspection without hand-written JOIN SQL for everyday operations.
- **Escape hatch retained:** The ORM's query builder still allows raw SQL and Core expressions for performance-critical analytics (e.g. the dashboard's aggregation queries).
- **Portability:** SQLAlchemy is database-agnostic at the code level, which keeps a future migration path open (e.g. to a managed cloud Postgres instance) without a rewrite.

1.3 Why a Relational Database (and not MongoDB)

VisionInspect AI's core entities have fixed, well-known schemas and strict referential relationships — a report always belongs to exactly one inspection, an inspection always belongs to exactly one image, an image always belongs to exactly one uploader. This is precisely the shape relational databases are built for. A document database such as MongoDB was considered but not selected, for the following reasons:

- **No referential integrity:** MongoDB has no native foreign-key enforcement — orphaned inspections or reports would have to be prevented entirely in application code instead of at the database layer.
- **Reporting-heavy workload:** Dashboard analytics (pass rate, defect counts grouped by date/category) are naturally expressed as SQL JOIN + GROUP BY queries, which are simpler and faster in a relational engine than in aggregation pipelines.
- **Transactional writes:** Inspection records must be atomically consistent with their parent image and child report; MongoDB's multi-document transactions exist but are heavier and less idiomatic than PostgreSQL's native ACID transactions.

- **Predictable schema:** The schema (users, images, inspections, reports) is stable and unlikely to need MongoDB's schema-less flexibility; a fixed schema instead gives the team compile-time and query-time safety.

1.4 File Storage Strategy: Uploads Folder vs. Database

Uploaded product images are written to disk under the backend's /uploads/ directory using a generated UUID filename, and only the resulting metadata — original filename, stored file path, uploader ID, and timestamp — is written to the uploaded_images table in PostgreSQL.

- **Why not store images in the DB:** Storing large binary image files directly inside PostgreSQL (as BYTEA/BLOB) bloats the database, slows down backups, and degrades query performance for unrelated tables.
- **Separation of concerns:** The filesystem (or later, an object store such as S3) is purpose-built for streaming large binary files efficiently, while PostgreSQL is purpose-built for structured, queryable metadata.
- **Serving efficiency:** FastAPI mounts /uploads/ as a static directory, so the frontend can render image thumbnails directly by URL without an extra database round-trip for the binary content.
- **Metadata as the bridge:** The filepath column in uploaded_images is the single link between the two storage systems — the database always knows where to find the physical file.

1.5 ACID Properties and Why They Matter Here

Property	Meaning	Why it matters for VisionInspect AI
Atomicity	A transaction either fully completes or fully rolls back.	If report generation fails after an inspection row is created, atomicity prevents a half-written inspection with no report.
Consistency	Every transaction leaves the database in a valid state.	Foreign key constraints guarantee an inspection can never reference a non-existent image.
Isolation	Concurrent transactions don't interfere with each other.	Two quality engineers uploading images simultaneously don't corrupt each other's metadata rows.
Durability	Once committed, data survives crashes/restarts.	A completed inspection result is never silently lost if the backend process restarts.

2. Database Architecture

VisionInspect AI follows a strictly layered architecture. Each layer has a single responsibility and only talks to the layer directly above or below it, which keeps the database access pattern predictable and testable.

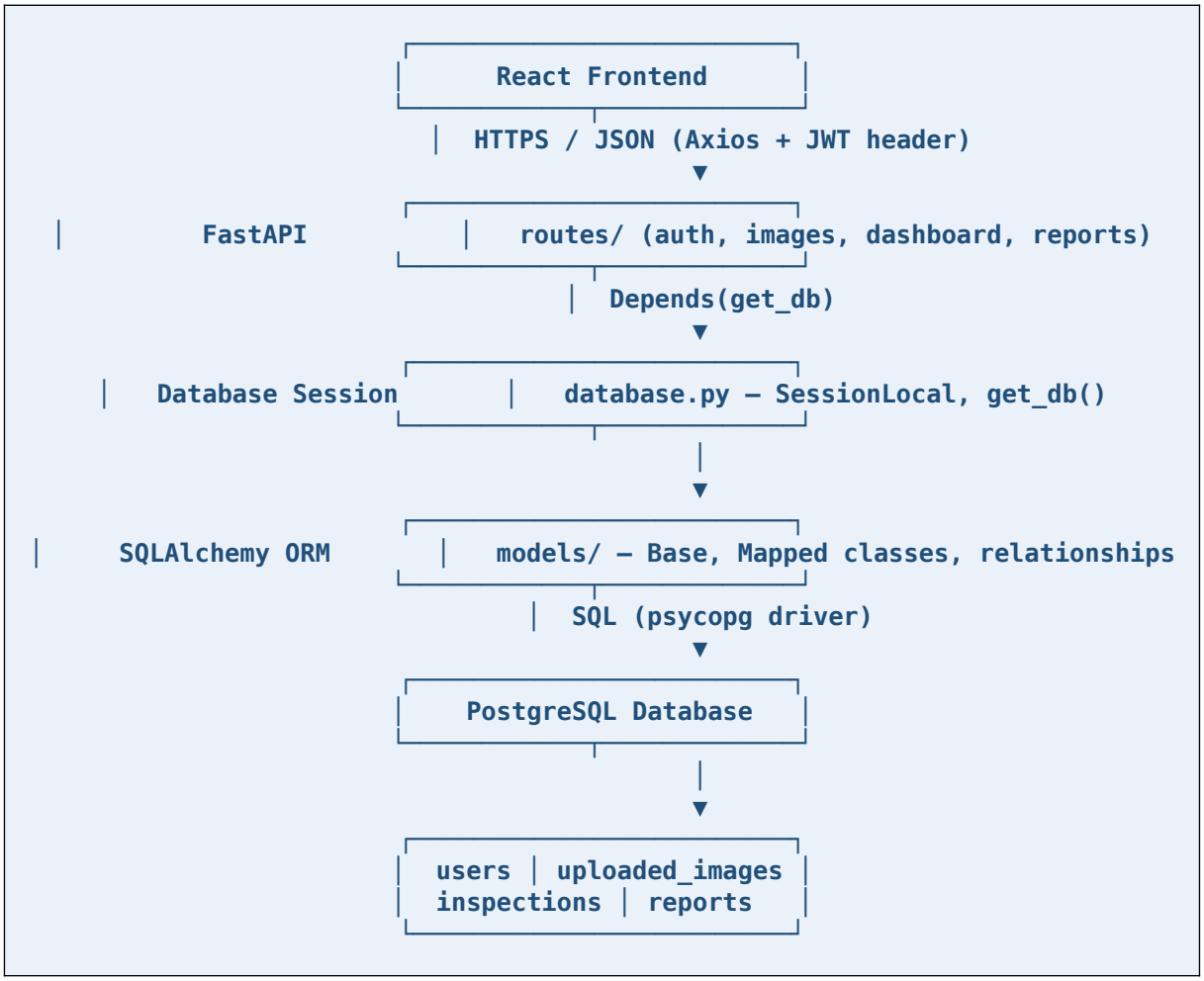


Figure 1: Layered request-to-storage architecture used across every API endpoint.

2.1 Layer Responsibilities

Layer	Responsibility
React Frontend	Collects user input, attaches the JWT to every request, renders data returned by FastAPI.
FastAPI Routes	Validates requests (Pydantic schemas), enforces auth/RBAC, calls service/model logic.
Database Session (database.py)	Owns the SQLAlchemy engine, creates a scoped Session per request, guarantees close()/rollback().
SQLAlchemy ORM (models/)	Maps Python classes to Postgres tables, defines relationships, generates SQL.
PostgreSQL	Executes SQL, enforces constraints, guarantees ACID transactions, persists data to disk.

3. Database File Structure

Every database-related concern is isolated into its own file inside the backend/ package. This keeps the ORM layer easy to navigate, test, and extend as new tables are added.

3.1 database.py — Engine & Session Factory

Purpose: creates the SQLAlchemy Engine (the connection pool to PostgreSQL) once at startup, and exposes a get_db() dependency that FastAPI injects into every route needing database access.

```
engine = create_engine(
    settings.DATABASE_URL,
    pool_size=10, max_overflow=20,
    pool_pre_ping=True, pool_recycle=1800,
)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

- Responsibilities: connection pooling, session lifecycle, health-checking stale connections (pool_pre_ping).
- Interacts with: every route file via FastAPI's Depends(get_db); models/base.py for metadata creation.
- Why it exists: centralizing engine creation avoids opening a new connection pool per request, which would exhaust PostgreSQL's max_connections under load.

3.2 models/base.py — Declarative Base

Purpose: defines the single DeclarativeBase subclass that every model inherits from, so SQLAlchemy can collect all table metadata in one MetaData registry.

```
class Base(DeclarativeBase):
    pass
```

- Responsibilities: acts as the shared registry SQLAlchemy uses to know which classes map to which tables.
- How SQLAlchemy uses it: Base.metadata.create_all(engine) introspects every subclass and issues the corresponding CREATE TABLE statements.

3.3 models/user.py — User Table Definition

```
class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    full_name: Mapped[str] = mapped_column(String(150), nullable=False)
    email: Mapped[str] = mapped_column(String(150), unique=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[str] = mapped_column(String(30), nullable=False,
default="quality_engineer")
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())

    uploaded_images: Mapped[list["UploadedImage"]] = relationship(
```

```
        back_populates="uploader", cascade="all, delete-orphan")
```

- Purpose: represents an operator account and its credentials/role.
- Interacts with: security.py (password hashing at write time), routes/auth.py (login/register), uploaded_images (one-to-many).

3.4 models/image.py — Uploaded Image Table Definition

```
class UploadedImage(Base):
    __tablename__ = "uploaded_images"
    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    filename: Mapped[str] = mapped_column(String(255), nullable=False)
    filepath: Mapped[str] = mapped_column(String(500), nullable=False)
    uploaded_by: Mapped[int] = mapped_column(ForeignKey("users.id",
ondelete="CASCADE"))
    uploaded_at: Mapped[datetime] = mapped_column(server_default=func.now())

    uploader: Mapped["User"] = relationship(back_populates="uploaded_images")
    inspections: Mapped[list["Inspection"]] = relationship(
        back_populates="image", cascade="all, delete-orphan")
```

- Purpose: logs every image captured on the production line, and the disk location where the binary file actually lives.
- Interacts with: images.py route (writes the row after saving the file to /uploads/), inspections table (one-to-many).

3.5 models/inspection.py — Inspection Table Definition

```
class Inspection(Base):
    __tablename__ = "inspections"
    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    image_id: Mapped[int] = mapped_column(ForeignKey("uploaded_images.id",
ondelete="CASCADE"))
    prediction: Mapped[str] = mapped_column(String(50), default="pending")
    confidence: Mapped[float] = mapped_column(default=0.0)
    severity: Mapped[str | None] = mapped_column(String(50), nullable=True)
    status: Mapped[str] = mapped_column(String(50), default="pending")
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())

    image: Mapped["UploadedImage"] = relationship(back_populates="inspections")
    report: Mapped["Report"] = relationship(
        back_populates="inspection", uselist=False, cascade="all, delete-orphan")
```

- Purpose: stores the AI model's classification output, confidence score, and status flag for a given image.
- Interacts with: the (future) prediction module that writes prediction/confidence; reports table (one-to-one via uselist=False).

3.6 models/report.py — Report Table Definition

```
class Report(Base):
    __tablename__ = "reports"
    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    inspection_id: Mapped[int] = mapped_column(
        ForeignKey("inspections.id", ondelete="CASCADE"), unique=True)
    report_path: Mapped[str] = mapped_column(String(500))
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())
```

```
    inspection: Mapped["Inspection"] = relationship(back_populates="report")
```

- Purpose: stores the generated compliance PDF's file path for a completed inspection.
- The unique=True constraint on inspection_id is what enforces the one-to-one relationship at the database level, not just in Python.

3.7 schemas/ — Pydantic Validation Layer

The schemas/ package (user.py, token.py, and similar files per resource) defines Pydantic models used purely at the API boundary — request bodies coming in and response bodies going out. They never touch the database directly; routes convert between ORM objects and Pydantic schemas explicitly. This keeps internal column names (e.g. password_hash) from ever leaking into an API response.

3.8 config.py — Database-Related Configuration

```
class Settings(BaseSettings):
    DATABASE_URL: str
    JWT_SECRET_KEY: str
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60
    class Config:
        env_file = ".env"

settings = Settings()
```

- Purpose: loads DATABASE_URL and related secrets from the .env file via Pydantic Settings, so credentials never live in source code.
- database.py imports settings.DATABASE_URL directly to build the Engine.

4. Database Design (Table-by-Table)

4.1 users

Purpose: stores every registered operator account and the role that determines their permissions across the platform.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique operator identifier.
full_name	VARCHAR(150)	NOT NULL	Operator's display name.
email	VARCHAR(150)	UNIQUE, NOT NULL	Login identifier.
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash — plaintext is never stored.
role	VARCHAR(30)	NOT NULL, DEFAULT 'quality_engineer'	admin / quality_engineer / factory_supervisor.
created_at	TIMESTAMP	SERVER_DEFAULT now()	Account creation time.

Primary Key: id. Foreign Keys: none (users is the root entity). Relationships: one-to-many with uploaded_images (uploaded_by).

Example Records

id	full_name	email	role
1	Akshaya Reddy	admin@visioninspect.com	admin
2	Test User	test@gmail.com	quality_engineer
3	test_	akkireddy@gmail.com	factory_supervisor

Used in the application to authenticate every request (JWT subject = user id), to attribute uploaded images to their uploader, and to drive the Admin Console's operator list and the RoleChecker/PermissionChecker guards.

4.2 uploaded_images

Purpose: logs metadata for every product image submitted through the Upload Images screen.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Unique image record identifier.
filename	VARCHAR(255)	NOT NULL	Original or display filename.
filepath	VARCHAR(500)	NOT NULL	Absolute path under /uploads/ on disk.
uploaded_by	INTEGER	FOREIGN KEY → users.id, ON DELETE CASCADE	Uploading operator.
uploaded_at	TIMESTAMP	SERVER_DEFAULT now()	Upload timestamp.

Primary Key: id. Foreign Keys: uploaded_by → users.id. Relationships: many-to-one with users; one-to-many with inspections.

Example Records

id	filename	uploaded_by	uploaded_at
1	Screenshot ...00-24-36.png	1	2026-07-09 12:48
4	Screenshot ...12-11-40.png	4	2026-07-10 17:21
5	Screenshot ...12-11-40.png	4	2026-07-10 18:35

Used to render the dashboard's image thumbnails (served statically from /uploads/), to schedule a pending inspection row immediately after a successful upload, and to trace any inspection back to the exact file and uploader that produced it.

4.3 inspections

Purpose: stores the AI model's per-image classification result — the core output of the inspection pipeline.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Unique inspection identifier.
image_id	INTEGER	FOREIGN KEY → uploaded_images.id, ON DELETE CASCADE	Image being inspected.
prediction	VARCHAR(50)	DEFAULT 'pending'	Normal / Defective / Pending.
confidence	FLOAT	DEFAULT 0.0	Model confidence score (0–1).
severity	VARCHAR(50)	NULLABLE	Defect severity, once classified.
status	VARCHAR(50)	DEFAULT 'pending'	pending / completed / failed.
created_at	TIMESTAMP	SERVER_DEFAULT now()	Inspection creation time.

Primary Key: id. Foreign Keys: image_id → uploaded_images.id. Relationships: many-to-one with uploaded_images; one-to-one with reports.

Example Records

id	image_id	prediction	confidence	status
1	4	Pending	0%	pending
2	5	Pending	0%	pending

Used to compute the dashboard's Total Inspected, Defects Detected, and Pass Rate metrics via aggregation queries, and to gate report generation — a report can only be created once an inspection reaches status = completed.

4.4 reports

Purpose: stores the file-system path of the generated compliance PDF for a completed inspection.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Unique report identifier.
inspection_id	INTEGER	FOREIGN KEY → inspections.id, UNIQUE, ON DELETE CASCADE	Parent inspection (1:1).
report_path	VARCHAR(500)	NOT NULL	Path to the generated PDF on disk.
created_at	TIMESTAMP	SERVER_DEFAULT now()	Report generation time.

Primary Key: id. Foreign Keys: inspection_id → inspections.id (unique, enforcing one-to-one). Relationships: one-to-one with inspections.

Used to power the Quality Reports screen — the frontend requests GET /reports, which joins reports to inspections and uploaded_images to display the image, prediction, and downloadable PDF together.

5. Table Relationships

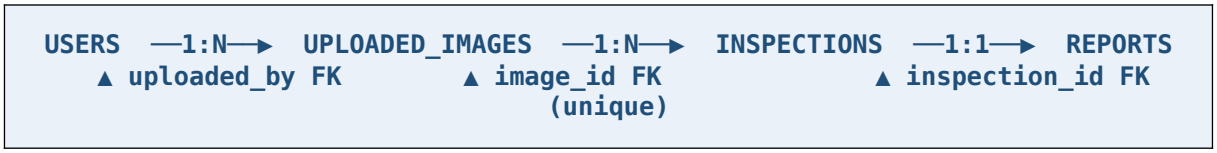


Figure 2: End-to-end referential chain from operator account to generated report.

5.1 User → Uploaded Images (One-to-Many)

A single user can upload many images, but each image belongs to exactly one uploader. This is modeled with a foreign key on the 'many' side (uploaded_images.uploaded_by) and a relationship() with a list type on the 'one' side.

```
# models/user.py (the "one" side)
uploaded_images: Mapped[list["UploadedImage"]] = relationship(
    back_populates="uploader", cascade="all, delete-orphan")

# models/image.py (the "many" side)
uploaded_by: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"))
uploader: Mapped["User"] = relationship(back_populates="uploaded_images")
```

5.2 Uploaded Image → Inspection (One-to-Many)

An image can undergo more than one inspection over its lifetime (e.g. a re-run after a model update), so this is also one-to-many, with the foreign key living on inspections.image_id.

5.3 Inspection → Report (One-to-One)

Every completed inspection produces exactly one compliance report. This is the one relationship in the schema that is deliberately restricted to one-to-one, enforced two ways at once:

- Database level: reports.inspection_id has a UNIQUE constraint, so PostgreSQL physically rejects a second report for the same inspection.
- ORM level: the relationship() on the Inspection side is declared with uselist=False, so SQLAlchemy exposes inspection.report as a single object instead of a list.

5.4 Key Concepts Used

Concept	Explanation
Primary Key	Uniquely identifies each row in a table (id in every table here).
Foreign Key	A column that references another table's primary key, enforcing that the referenced row must exist.
Cascade Delete	ondelete="CASCADE" — deleting a parent row (e.g. a user) automatically deletes its dependent rows (their images, inspections, reports), preventing orphaned data.
back_populates	Links two relationship() declarations so that setting one side (e.g. image.uploader) automatically keeps the other side (user.uploaded_images) in sync in Python memory.
relationship()	A SQLAlchemy construct that lets Python code navigate foreign keys as object attributes instead of writing manual JOINs for everyday access.
Mapped[]	The typing wrapper (SQLAlchemy 2.0) that gives each column a real Python type,

	enabling IDE autocomplete and static type checking.
mapped_column()	Declares the actual database column — its SQL type, constraints, defaults, and foreign key — attached to a Mapped[] attribute.

6. SQLAlchemy Implementation

6.1 Base

Base is the shared DeclarativeBase subclass every model inherits from. It maintains the MetaData registry that records every table, column, and constraint defined across the models/ package, and it's what Base.metadata.create_all(engine) reads when generating the schema.

6.2 Engine

The Engine is SQLAlchemy's core interface to the database — it owns the connection pool and knows how to speak the PostgreSQL wire protocol via the psycopg driver. It is created exactly once, at import time in database.py, and reused for the lifetime of the application process.

6.3 Session

A Session is a single unit-of-work: it tracks every object added, modified, or deleted during a request, and issues the corresponding INSERT/UPDATE/DELETE statements only when db.commit() is called. FastAPI's get_db() dependency opens a fresh Session per HTTP request and always closes it afterward, so sessions are never shared across requests.

6.4 Mapped and mapped_column()

Together, Mapped[int] and mapped_column(primary_key=True) declare one column with full static typing: Mapped[] tells Python (and your IDE) the attribute's type, while mapped_column() tells SQLAlchemy the actual SQL column definition — type, nullability, default, foreign key, uniqueness.

6.5 relationship()

relationship() does not create a database column; it creates a Python-level convenience so related rows can be accessed as attributes (e.g. image.inspections) instead of writing a JOIN by hand. Under the hood, SQLAlchemy still issues a SELECT with a WHERE clause on the foreign key when the relationship is accessed (unless eagerly loaded).

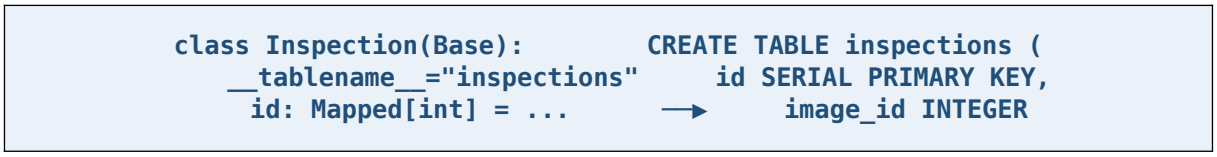
6.6 ForeignKey

ForeignKey("uploaded_images.id") tells PostgreSQL that this column's value must match an existing id in uploaded_images, or be NULL if nullable. This is the mechanism that makes cascade deletes and referential integrity possible.

6.7 server_default and func.now()

server_default=func.now() pushes the default value's computation to PostgreSQL itself (via the SQL NOW() function) rather than to Python. This guarantees the timestamp reflects the database server's clock at the moment of insertion, even if multiple application servers with slightly different clocks are writing concurrently.

6.8 From Python Class to PostgreSQL Table



```
    image_id: Mapped[int] = ...          REFERENCES uploaded_images(id)
          prediction: Mapped[str] = ...       ON DELETE CASCADE,
        ...                                 prediction VARCHAR(50),
                                                      ...
                                                    );
```

Figure 3: Base.metadata.create_all(engine) translates each Mapped class into the equivalent DDL.

7. Database Flow

The following sequence traces a single image from upload through to a downloadable report, showing exactly where each database write occurs.

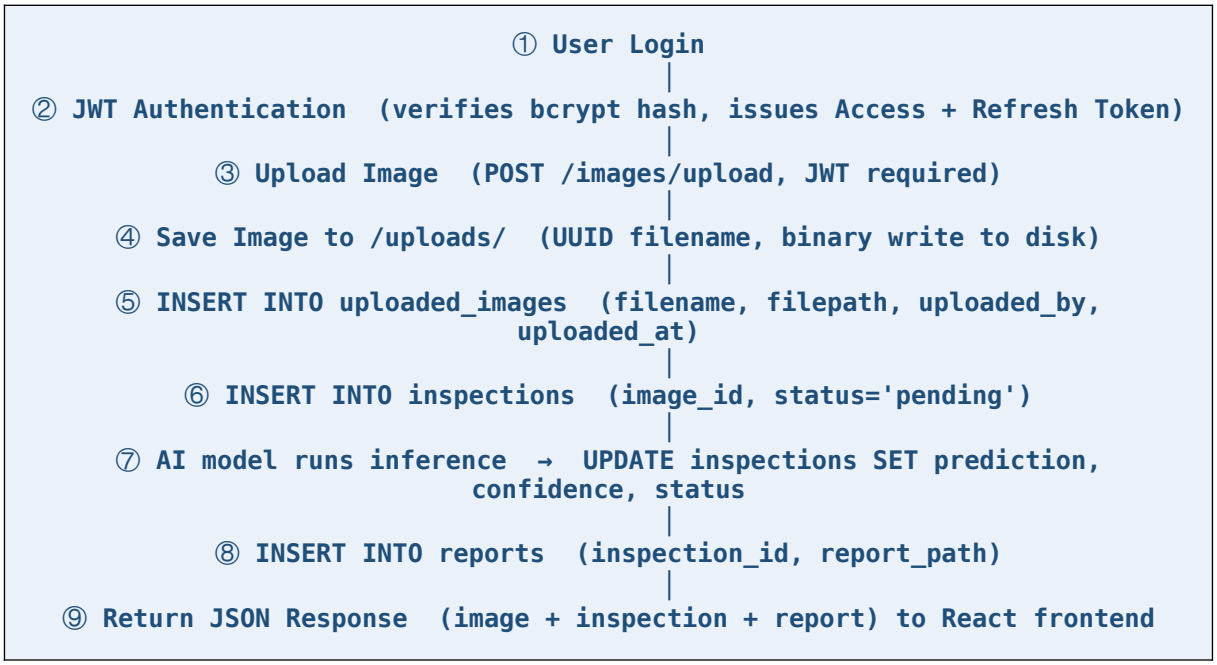


Figure 4: Complete request-to-persistence flow for one inspection cycle.

8. ER Diagrams

Two complementary ER diagrams were produced for VisionInspect AI: a color-coded conceptual diagram showing entity attributes and Crow's Foot cardinality, and a structural diagram emphasizing primary/foreign keys and relationship cardinalities.

8.1 Conceptual ER Diagram (Crow's Foot Notation)

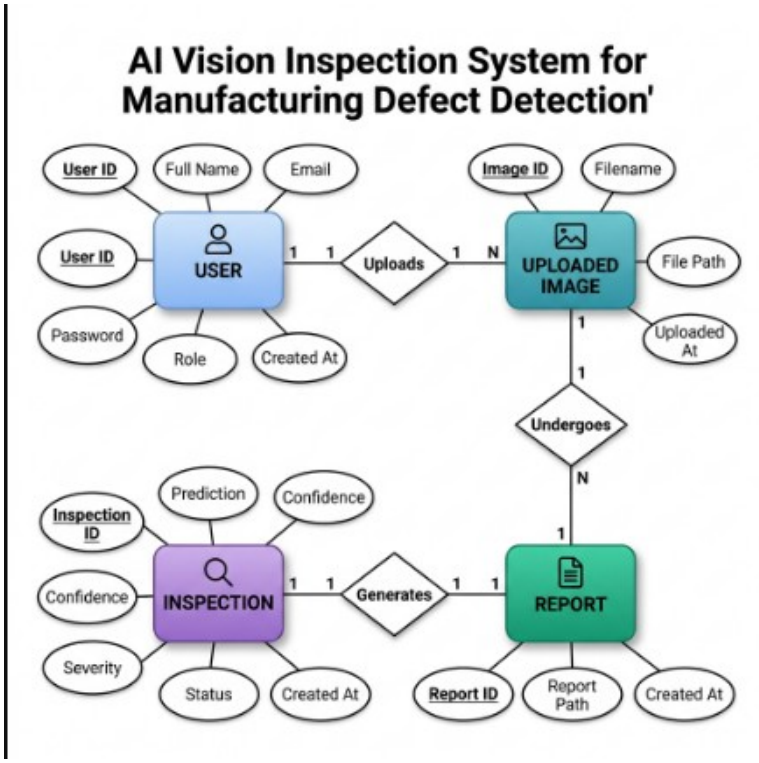


Figure 5: Entity-relationship diagram for VisionInspect AI — USER, UPLOADED IMAGE, INSPECTION, and REPORT, with attributes and Crow's Foot cardinality (1:1, 1:N) on each relationship.

8.2 Structural Relationship Diagram

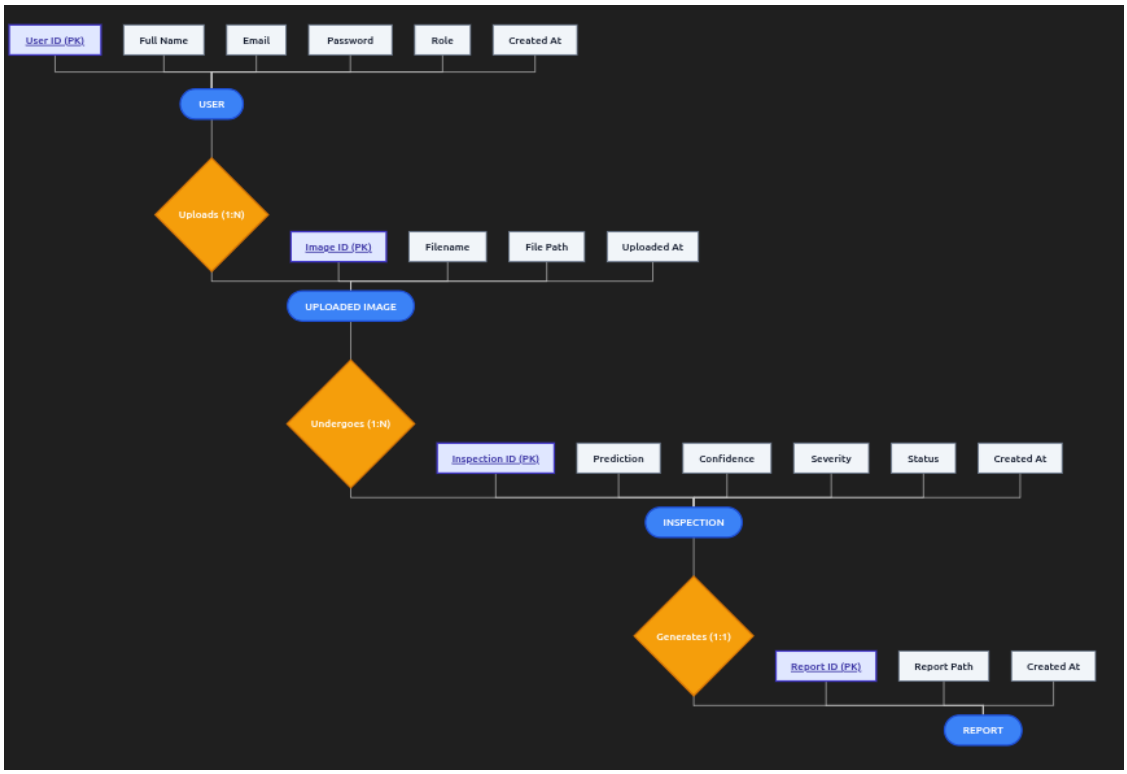


Figure 6: Structural view of the same four tables, showing each column beneath its owning entity and the Uploads (1:N) → Undergoes (1:N) → Generates (1:1) relationship chain.

8.3 Reading the Diagrams

- A single bar (1) at the USER end of Uploads means one user; the crow's foot (N) at UPLOADED IMAGE means many images per user.
- The Undergoes relationship is 1:N — an image may be inspected more than once (e.g. re-inspection after a model update).
- The Generates relationship is capped at 1:1 by the UNIQUE constraint on reports.inspection_id — exactly one report per inspection.

9. Database Normalization

9.1 First Normal Form (1NF)

A table is in 1NF if every column holds a single atomic value and there are no repeating groups. All four tables satisfy this: for example, uploaded_images never stores multiple filenames in one row — each upload is its own row, and every column (filename, filepath, uploaded_by) holds one indivisible value.

9.2 Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and every non-key column depends on the whole primary key. Because every table here uses a single-column surrogate key (id), there is no composite key to create partial dependency — so 2NF is automatically satisfied. For example, in inspections, prediction and confidence depend only on id, not on some subset of a composite key.

9.3 Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and no non-key column depends on another non-key column (no transitive dependency). This was checked table by table:

- users: role and password_hash each depend only on id, not on each other.
- uploaded_images: filepath depends on id, not on uploaded_by — the uploader doesn't determine the file path.
- inspections: confidence and severity are independent outputs of the model, both depending only on id, not on each other.
- reports: report_path depends only on id (and the enforced 1:1 inspection_id), with no other transitive column to create a dependency.

Conclusion: the schema is in 3NF. Data such as a user's full_name is stored exactly once (in users) and referenced everywhere else via uploaded_by, so an update to an operator's name never requires touching image, inspection, or report rows — eliminating update anomalies.

10. SQL Queries

Representative queries actually used (or directly usable) by the VisionInspect AI backend, grouped by operation type.

10.1 SELECT

```
-- All images uploaded by a specific operator
SELECT id, filename, uploaded_at
FROM uploaded_images
WHERE uploaded_by = 4
ORDER BY uploaded_at DESC;
```

10.2 INSERT

```
INSERT INTO uploaded_images (filename, filepath, uploaded_by)
VALUES ('bottle_007.png', '/uploads/8f69adb2.png', 4)
RETURNING id;
```

10.3 UPDATE

```
-- Write back the AI model's result once inference completes
UPDATE inspections
SET prediction = 'Defective', confidence = 0.942, status = 'completed'
WHERE id = 7;
```

10.4 DELETE

```
-- Cascade-deletes the user's images, inspections, and reports automatically
DELETE FROM users WHERE id = 9;
```

10.5 JOIN (Reports screen)

```
SELECT r.id, r.report_path, i.prediction, i.confidence, img.filename
FROM reports r
JOIN inspections i ON i.id = r.inspection_id
JOIN uploaded_images img ON img.id = i.image_id
ORDER BY r.created_at DESC;
```

10.6 COUNT / GROUP BY (Dashboard analytics)

```
SELECT status, COUNT(*) AS total
FROM inspections
GROUP BY status;

-- Pass rate calculation
SELECT
  ROUND(100.0 * SUM(CASE WHEN prediction = 'Normal' THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0), 2) AS pass_rate_pct
FROM inspections
WHERE status = 'completed';
```

10.7 ORDER BY + LIMIT (Recent inspections widget)

```
SELECT img.filename, i.prediction, i.confidence, i.status, i.created_at
FROM inspections i
JOIN uploaded_images img ON img.id = i.image_id
ORDER BY i.created_at DESC
LIMIT 5;
```

11. Database Security

11.1 Password Hashing

Passwords are never stored as plaintext. passlib's bcrypt scheme hashes each password with a per-password salt before it is written to users.password_hash; login compares the submitted password against the stored hash using bcrypt's constant-time verify(), never a plaintext comparison.

11.2 JWT Authentication

After a successful login, the backend issues a signed JWT Access Token (60 minute expiry) and Refresh Token (7 day expiry). Every protected route requires a valid, unexpired Access Token in the Authorization header; the token's subject claim carries the user's id, which routes use to attribute uploads and inspections to the correct operator without re-querying credentials.

11.3 Role-Based Access Control (RBAC)

RoleChecker and PermissionChecker dependencies run before a route's business logic executes, comparing the authenticated user's role against the permission map defined in rbac.py. An unauthorized request never reaches the database layer at all — it is rejected with HTTP 403 at the routing layer.

11.4 Foreign Key Constraints & Data Integrity

Every child table (uploaded_images, inspections, reports) declares a NOT NULL foreign key with ON DELETE CASCADE. This guarantees the database itself — not just application code — rejects an inspection pointing at a non-existent image, and automatically cleans up dependent rows if a parent is removed.

11.5 Transactions

Each request's Session wraps its writes in a single transaction; if any step fails (e.g. saving the file succeeds but the metadata INSERT fails), the Session is rolled back so the database is never left with a metadata row pointing at a file that doesn't exist, or vice versa.

12. Project Database Workflow

Figure 7: End-to-end request lifecycle for any database-backed API call.

Stage	What happens
React	Builds the request (e.g. image upload form data) and attaches the JWT.
FastAPI	Receives the HTTP request, validates it against a Pydantic schema.
Routes	Runs auth/RBAC dependencies, then calls the relevant service/model logic.
Database Session	get_db() supplies a Session scoped to this single request.
Models	ORM classes translate Python objects into SQL INSERT/SELECT/UPDATE statements.
PostgreSQL	Executes the SQL, enforces constraints, commits or rolls back.
Response	The route serializes the resulting ORM object(s) back into JSON for React.

13. Database Advantages

- **Scalability:** A normalized four-table schema with proper foreign keys supports growing to many more operators, cameras, and daily inspections without a redesign; PostgreSQL itself scales vertically and, with read replicas, horizontally for read-heavy dashboards.
- **Maintainability:** Each table lives in its own model file with a single clear responsibility, so adding a new column or table (e.g. a defect_types lookup table) touches a small, well-isolated part of the codebase.
- **Data Integrity:** Foreign keys, NOT NULL constraints, and cascade rules are enforced by PostgreSQL itself, so invalid states (an inspection with no image, a report with no inspection) are impossible regardless of application-level bugs.
- **Security:** Bcrypt password hashing, JWT-based auth, and RBAC guards mean the database is never reachable with an unauthenticated or under-privileged request.
- **Performance:** Indexed primary/foreign keys and connection pooling keep dashboard aggregation queries fast even as the inspections table grows into the hundreds of thousands of rows.